

Deep Reinforcement Learning and Applications

Modern algorithms and real-world challenges — Lecture 4/4

Etienne Chassaing

CentraleSupélec

1. Deep Q-Learning
2. Actor-Critic Methods
3. Modern Challenges
4. Case Studies

By the end of this lecture, you should understand:

1. **Deep Q-Networks (DQN)** — extend Q-learning to continuous/large state spaces using neural networks
2. **Actor-Critic methods** — learn both policy (actor) and value (critic) simultaneously
3. **Modern challenges** — alignment, exploration in high-dimensional spaces, simulation-to-reality gap
4. **Case studies** — application to optimal control problems (heat pipe, robotics, etc.)
5. **Practical lessons** — what works in practice, common pitfalls, when to use each algorithm

Context: Lectures 1–3 covered tabular methods. Here, we scale RL to real-world problems using neural networks and explore why deployment is hard.

Goals of today's session

Complete this slide with session-specific learning objectives

Lectures 1–3: Tabular RL

- Finite state/action spaces
- Tables to store V or Q
- Convergence proofs available
- Works for small problems

Challenge: Neural networks break many nice properties of tabular RL (convergence, stability). We need tricks (experience replay, target networks, etc.).

Lecture 4: Deep RL

- Continuous/very large state spaces
- Neural networks to approximate V or Q
- Heuristics work but no guarantees
- Real-world applications

1. Deep Q-Learning
2. Actor-Critic Methods
3. Modern Challenges
4. Case Studies

Deep Q-Networks (DQN)

Main idea: Replace lookup table $Q(s, a)$ with a neural network $Q(s, a; \theta)$.

Naive approach fails:

- TD updates are highly correlated (sequential states)
- Bootstrapping on unstable target network diverges
- Learning is inefficient

DQN tricks:

1. **Experience Replay:** Store (s, a, r, s') in buffer, sample random minibatches to break correlation
2. **Target Network:** Separate network $Q(s', a'; \theta^-)$ for computing targets, update slowly
3. **Clipped Rewards:** Normalize rewards for stability

Update rule:

$$Q(s, a; \theta) \leftarrow Q(s, a; \theta) + \alpha[r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)]$$

Experience Replay & Target Networks

Why these tricks matter:

Experience Replay:

- Breaks temporal correlation
- Allows reuse of old data
- More sample-efficient

Implementation:

- Store last N transitions
- Sample B random batches
- Update network on batch

Target Network:

- Q network updates online
- Q^- network updates slowly (every C steps)
- Stabilizes target

Why works:

- Without it: target shifts as Q learns
- Result: oscillation or divergence
- Lagged target = more stable training

1. Deep Q-Learning
2. Actor-Critic Methods
3. Modern Challenges
4. Case Studies

Actor-Critic Architecture

Idea: Learn two networks simultaneously

- **Actor** $\pi(a|s; \phi)$: policy network (what action to take?)
- **Critic** $V(s; \theta)$: value network (how good is this state?)

Algorithm:

1. Actor takes action according to $\pi(a|s; \phi)$
2. Observe (s, a, r, s')
3. Critic evaluates: TD error $\delta = r + \gamma V(s'; \theta) - V(s; \theta)$
4. Update critic: $\theta \leftarrow \theta + \alpha \delta \nabla_{\theta} V(s; \theta)$
5. Update actor: $\phi \leftarrow \phi + \beta \delta \nabla_{\phi} \log \pi(a|s; \phi)$

Advantage:

- Lower variance than policy gradient (critic reduces noise)
- Better than Q-learning for continuous actions (can parameterize policy directly)

1. Deep Q-Learning
2. Actor-Critic Methods
- 3. Modern Challenges**
4. Case Studies

The Alignment Problem

Definition: Agent optimizes reward function but deviates from designer intent.

Classic examples:

- Atari: Agent learns to exploit glitches instead of playing well
- Grasping robot: Moves camera instead of moving arm
- Dialogue agent: Manipulates rewards through hacking

Root cause: Reward function captures only measurable proxy, not true objective.

Partial solutions:

- Better reward design (harder than it sounds)
- Constrained optimization (stay close to safe region)
- Human feedback (RLHF)
- Interpretability (understand what agent learned)

Open problem: No silver bullet for alignment in open-ended environments.

Simulation-to-Reality Gap

Problem: Agent trained in simulation performs poorly on real robot.

Sources of gap:

- Physics errors in simulator (friction, inertia, collisions)
- Sensor noise and delays not modeled
- Actuator characteristics (dead zones, limits)
- Lighting, textures, camera calibration

Solutions:

1. **Domain Randomization:** Vary simulation parameters during training
 - ▶ Force agent to learn robust policy across many conditions
 - ▶ Works surprisingly well in practice
2. **Sim-to-Real Transfer:** Fine-tune on real robot with RL
3. **Model Adaptation:** Learn correction function from real data

Reality: Most production systems combine simulation + real-world data.

1. Deep Q-Learning
2. Actor-Critic Methods
3. Modern Challenges
4. Case Studies

Case Study 1: Heat Pipe Control

Problem: Control temperature in a heat pipe system (complex nonlinear dynamics).

Why RL?

- No accurate physics model
- Tuning PID gains is tedious
- System has constraints (power, safety bounds)
- Data-driven approach preferred

MDP Formulation:

- State: temperature + rate of change
- Action: heater power (0–100%)
- Reward: $-(|T - T_{\text{target}}| + 0.1 \times \text{power}^2)$

Results:

- Q-Learning: Fast learning, may overshoot
- SARSA: More stable, conservative
- Actor-Critic: Smooth control, best long-term

Key lesson: Different algorithms suit different objectives. Safety-critical systems favor conservative methods.

Ref: https://etiennechassaing.com/book/case-studies/heat_pipe_control/

Key Takeaways

- **Deep RL:** Scales to continuous/large spaces but needs tricks (replay, target nets)
- **Actor-Critic:** Balances exploration and stability via separate policy/value networks
- **Alignment:** Reward design is hard; misspecified rewards cause agent to exploit
- **Sim-to-Real:** Domain randomization helps; real-world fine-tuning often necessary
- **Practical:** Choose algorithm based on problem structure (safety, sample efficiency, interpretability)
- **Future:** More reliable sim-to-real, better alignment, sample-efficient learning remain open

Thank you!

Questions?

`etienne.chassaing@centralesupelec.fr`



Course materials and references:

`github.com/cetiennec/cours-pdf`



Next: Independent projects using RL!